The adaptive DIPF policy (v14)

How a 40-number learned policy retunes the potential field every 0.1 s, what it learned, and how it compares with the best fixed parameter set

1. Summary

The DIPF potential field drives the ego exactly as before: every acceleration is the clamped negative gradient of the potential. The adaptive policy changes four of the potential's parameters while the ego drives, based on nine measurements of the surrounding traffic. It was trained with CMA-ES on closed-loop segment success, on top of the **best fixed parameter set** that the same search could find (v14/base).

on 300 fresh test scenarios × 3 futures (900 paired runs)	**success**	**progress**
v12 library set 2 (the v13 base)	0.588	0.750
trained fixed set (v14/base, all 7 parameters tuned)	0.646	0.780
adaptive policy (v14/adaptive)	**0.714**	**0.812**

Adaptive vs the best fixed set: **+6.9 points** (runs only adaptive succeeded 182, only the fixed set 120; McNemar p = 0.0004). About one in five of the fixed set's failures becomes a success. The policy's offsets alone, without the adapting, are no better than the fixed set, so the gain comes from adapting during the run.

2. Where the policy sits in the controller

Every 0.1 s the controller computes $U = w_{dest}\cdot U_{dest} + w_{lane}\cdot U_{lane} + w_{obs}\cdot \Sigma U_{obs}$, its gradient, and $a = \text{clamp}(-\nabla U)$: +3.2 / −9 m/s² along the road, ±3 m/s² across, speed ≤ 34 m/s, sideways speed ≤ 3 m/s and ≤ $v_x\cdot\tan 15°$. The policy runs just before this and only sets parameters. It never outputs an acceleration, so a mistake can reshape the field but cannot command an arbitrary manoeuvre.

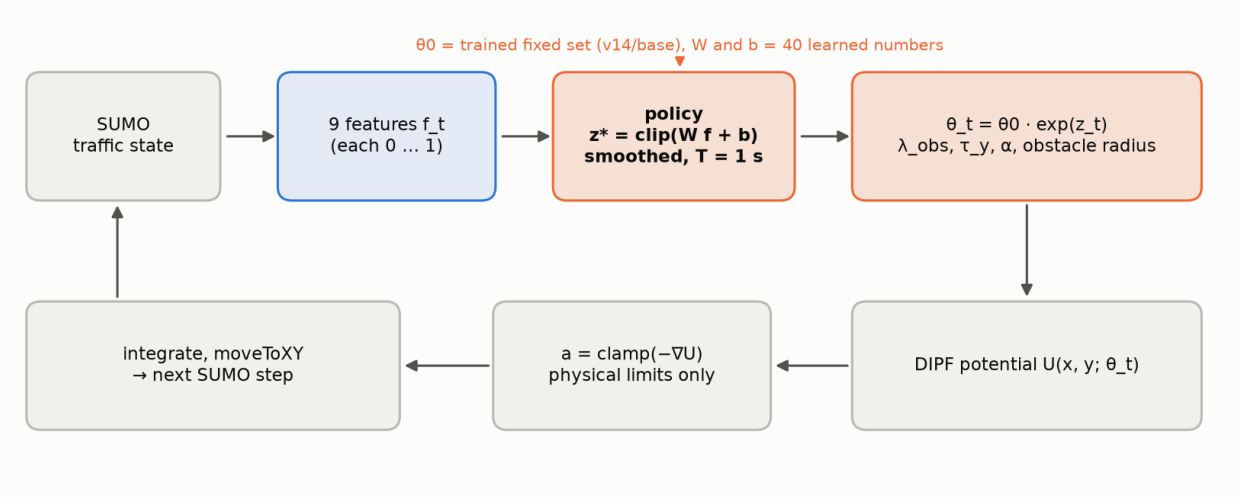


Figure 1. One control step. Orange: the policy; the rest is the unchanged DIPF controller.

3. Inputs: nine dynamics features

The 80-value traffic encoding is condensed into nine numbers, each scaled to [0, 1]. They describe how threatening or crowded the situation is, not where vehicles are (the potential already handles direction), and none of them changes when left and right are swapped, matching the symmetric controller.

feature	**definition**	**large (→1) when**
ttc_front	$1 - e^{-1/\text{TTC}}$, vehicle in the ego's path ahead	closing in fast ahead (TTC < 1 s)
ttc_side_front	same, worst of front-left / front-right	closing in on a diagonal vehicle

feature	definition	large (→1) when
ttc_rear_side	same, worst of rear, rear-left/right, alongside	something closing from behind or the side
drac_front	$1 - e^{-DRAC/3}$, DRAC = closing² / (2·gap)	hard braking needed
gap_front	$e^{-gap/20\ m}$ (0 if nothing ahead)	a vehicle close ahead
gap_side_front	same, closest of front-left / front-right	a vehicle close diagonally ahead
tight_lateral	$e^{-room/1\ m}$, room = smallest road-edge clearance or side gap	no room to swerve
speed	v_x / 34 m/s	driving fast
density	$1 - e^{-n/10}$, n = vehicles within ±50 m	crowded

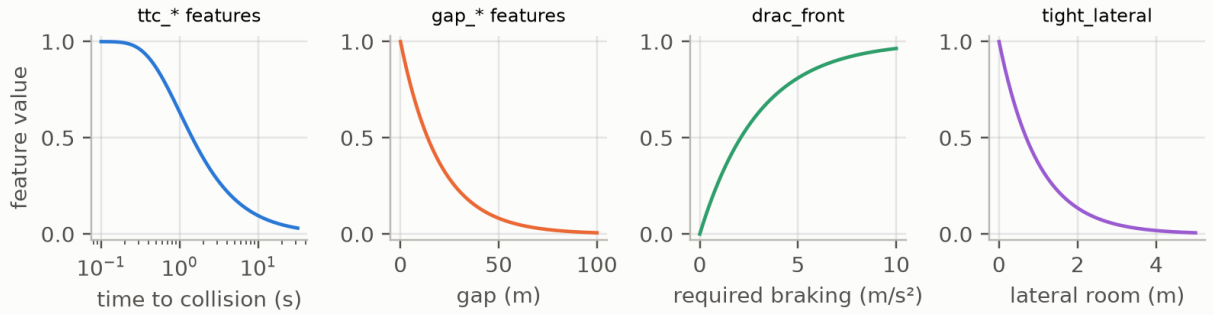


Figure 2. The transforms map unbounded quantities into [0, 1]; "nothing there" and "far away" both give 0, and each saturates where it stops mattering.

4. From features to parameters

```
z*_t = clip(W·f_t + b, −ln 10, +ln 10)      W: 4×9, b: 4 → 40 numbers
z_t  = z_(t−1) + (Δt/T)·(z*_t − z_(t−1))    T = 1 s, Δt = 0.1 s; first step z = z*
p_t  = clamp(p0·exp(z_t), limits)           p: λ_obs, τ_y, α, obstacle radius
```

Each adapted parameter gets one exponent: a weighted sum of the features plus an offset. The exponential makes changes multiplicative (natural for parameters spanning orders of magnitude) and keeps them positive; the clip limits each parameter to ×0.1 … ×10 of its base value; the absolute limits are the base search's ranges. Smoothing moves the exponent 10 % of the way to its target each step, so a parameter follows a sudden change within about a second instead of jumping. With W = 0 and b = 0 the policy reproduces the fixed set exactly (checked run by run).

adapted parameter	base p0 (trained fixed set)	limits	role in the potential
λ_obs (lambda_obs)	190.7	0.01 … 10000	strength of every obstacle's repulsion, U_obs = λ / k′
τ_y (tau_y_base)	0.3776	0.01 … 20	lateral scale in k′: larger → vehicles to the side repel more
α (alpha)	0.02091	0.01 … 100	how strongly τ grows with the relative velocity Δv
obs_radius_ahead	97.48 m	10 … 250 m	how far ahead obstacles are considered

The other parameters stay at the trained fixed set's values: s_x = 0.000992, δ_{lane} = 100, y_{margin} = 0.00542; τ_x = 1. Sign constraint: the weights from ttc_front, ttc_side_front and drac_front to λ_{obs} may not be negative, so repulsion can only grow with those dangers.

5. What the policy learned

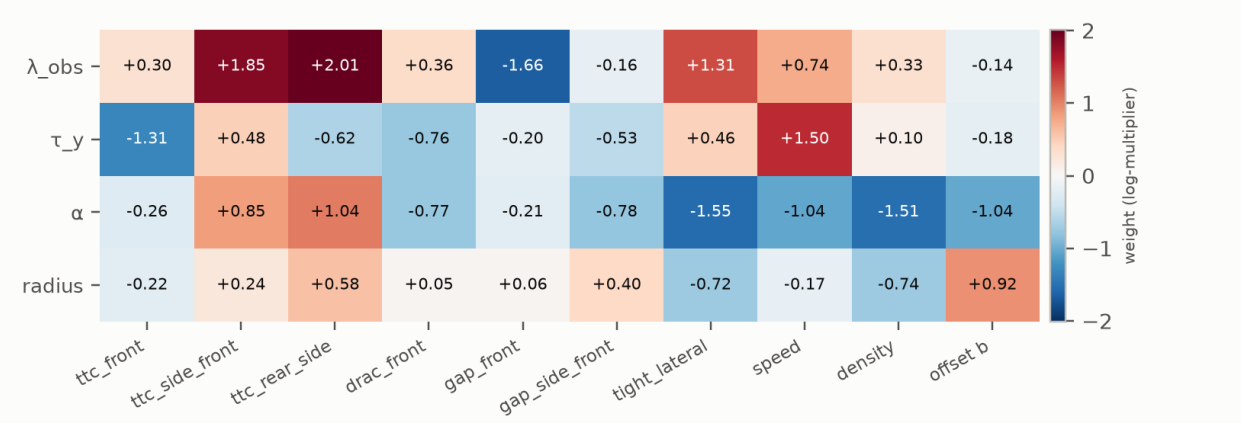


Figure 3. The 40 learned numbers. Each weight is a log-multiplier per unit feature: +1 means the parameter is multiplied by e ≈ 2.7 when that feature goes from 0 to 1.

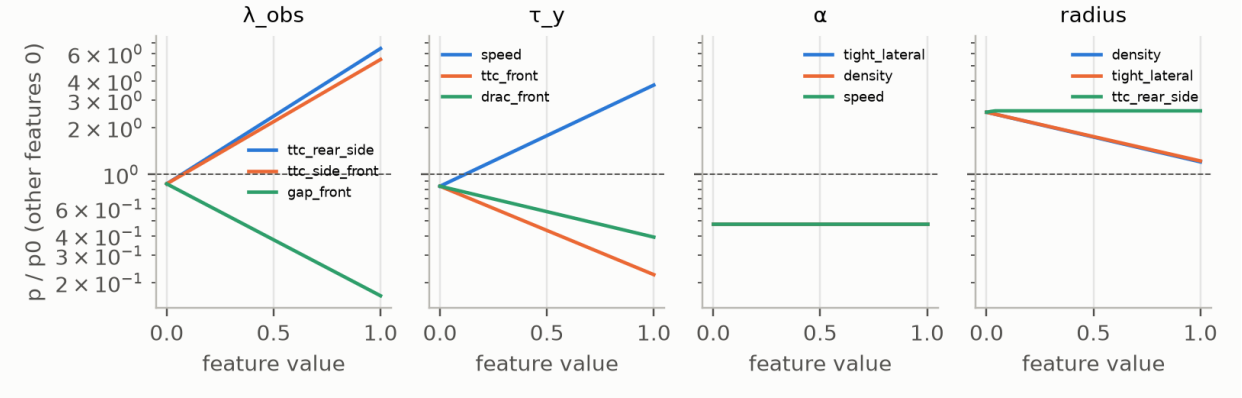


Figure 4. Each parameter's multiplier as one of its three strongest features rises from 0 to 1 (all other features 0, offset included). Real contexts combine several features.

Read together, the weights describe one strategy: **relaxed by default, sharp reactions to threats from the side and behind**.

λ_{obs}, repulsion strength

- Calm offset ×0.87.
- Rises with something closing from behind or the side (ttc_rear_side +2.0) or diagonally ahead (ttc_side_front +1.85), with little lateral room (+1.3) and with speed (+0.7).
- Falls when a vehicle is close ahead but not closing (gap_front −1.7): the ego follows calmly instead of being pushed sideways by a car that is no threat.

τ_y, lateral reach

- Rises with speed (+1.5): at 20 m/s the lateral influence zone is about twice as wide.
- Falls when closing in ahead (ttc_front −1.3): vehicles beside the ego count less, leaving room to swerve around the obstacle ahead.

α, velocity dependence of the reach

- The offset (×0.35) puts it below its 0.01 floor in calm traffic, and the strongest weights (density −1.5, tight_lateral −1.6, speed −1.0) push it further down. In practice α is pinned at its lower limit in 100% of control steps (Figure 6): the policy effectively chose the smallest allowed α, i.e. obstacles' reach barely depends on relative speed.

Obstacle radius, look-ahead

- Offset ×2.51: the policy looks much further ahead than the fixed set (97 m). Measured median ×1.79 (≈ 174 m).

- Shorter in dense traffic and when lateral room is tight (−0.7 each), so distant vehicles do not distract it in a crowd.

6. The policy in action

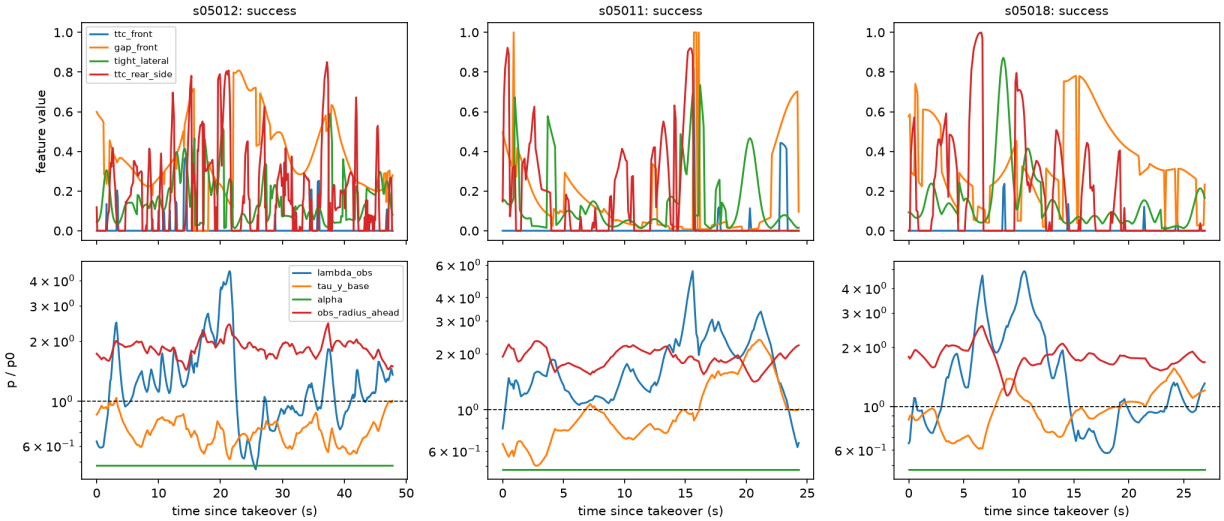


Figure 5. Three test runs (all successes). Top: four of the features. Bottom: the parameter multipliers p/p0 (log scale). λ_obs (blue) spikes to ×3–5 when ttc_rear_side or tight_lateral rise and relaxes afterwards; α (green) is flat at its 0.01 floor.

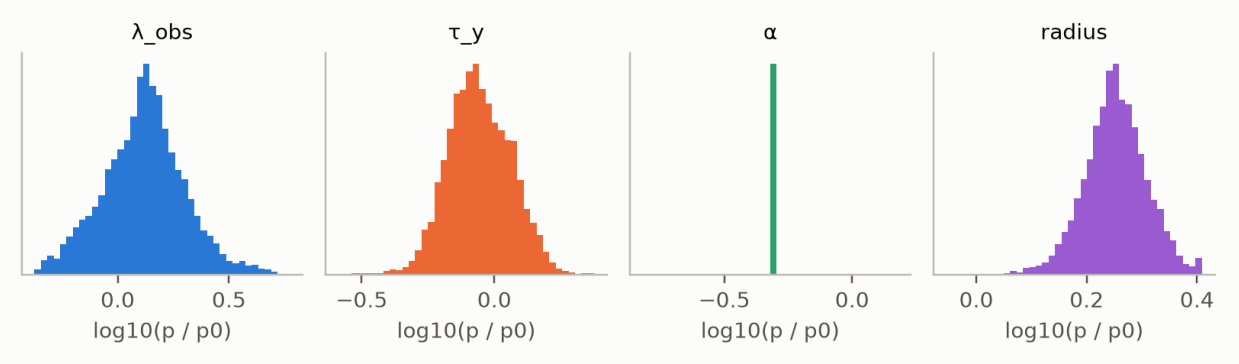


Figure 6. How far the parameters move: all 10,117 control steps of 40 test runs.

parameter	median	5 % … 95 %	steps at a limit
λ_obs	×1.33	×0.63 … ×2.67	0%
τ_y	×0.87	×0.57 … ×1.39	0%
α	×0.48	×0.48 … ×0.48	100%
radius	×1.79	×1.43 … ×2.21	0%

Worked examples

Multipliers computed with the real policy code for typical situations (target values, before smoothing; features not listed are 0):

situation	λ_obs	τ_y	α	radius
Calm: nothing near, 15 m/s, light traffic	×1.31	×1.67	×0.48	×1.93
Following a car 10 m ahead at equal speed	×0.50	×1.50	×0.48	×1.81
Car closing from rear-left, TTC 1 s	×4.88	×1.15	×0.48	×2.52
Diagonal car ahead closing, TTC 1 s, tight sideways	×9.21	×2.15	×0.48	×1.52
Closing on the car ahead, TTC 1.5 s, braking 2 m/s²	×0.76	×0.56	×0.48	×1.66

The same ego behaves differently in each. Behind a non-threatening car repulsion is lowered, so it follows calmly. When a vehicle closes from the side or behind, repulsion jumps several-fold (×5–9), pushing the ego away or slowing it until the threat passes. Closing on the

car directly ahead is different: repulsion stays slightly *below* the fixed set (the 'vehicle close ahead' weight outweighs the closing weights) and the lateral reach narrows. The policy leaves head-on closing mostly to the field's existing shape and concentrates its reactions on side and rear threats.

7. How it was trained

There is no correct parameter value at time t to learn from (many values work and a choice shows its effect only seconds later), so the policy was trained directly on what matters:

```
J(W, b) = success rate + 0.1 · mean progress
          over 100 train scenarios × 2 futures, takeover (x ≈ 100 m) → x = 400 m
```

CMA-ES (an evolution strategy) searched the 40 numbers, starting at 0 (= the fixed set): 30 generations × 15 candidates, every candidate scored on the same 200 runs. Every 4 generations the search mean was scored on 100 validation scenarios × 3 futures, and the final policy is the validated one with the highest J: the search mean of generation 12 (validation success 0.730 vs 0.647 for the fixed set). The fixed set itself was trained by the same CMA-ES over all 7 parameters with the same budget (450 candidates).

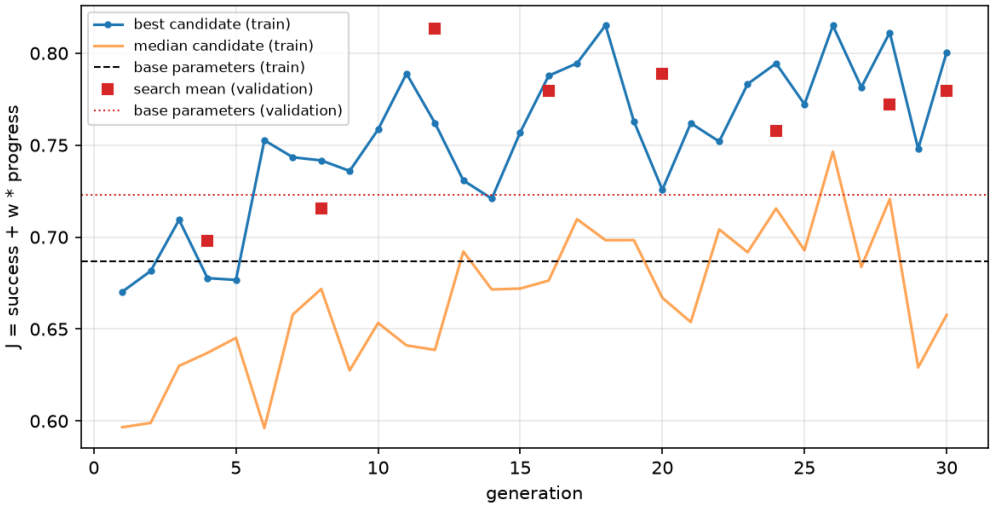


Figure 7. Training progress. Squares: the search mean on validation (the selection criterion); dotted: the fixed set on validation. Training J is optimistic (best of 15).

8. Results

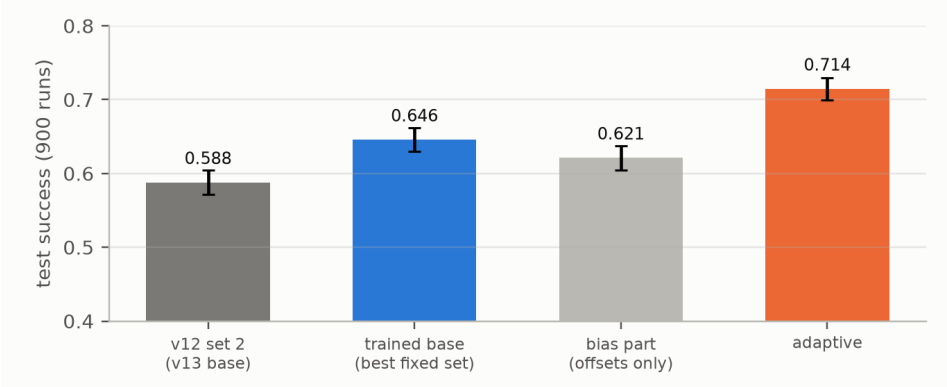


Figure 8. Test success, 300 fresh scenarios × 3 futures (±1 standard error).

comparison (same 900 runs)	difference	only A / only B	McNemar p
adaptive vs trained fixed set	+6.9 ± 2.2	182 / 120	0.00043
trained fixed set vs v12 set 2	+5.8 ± 2.5	206 / 154	0.0071
adaptive vs its offsets only	+9.3 ± 2.4	201 / 117	2.9e-06

comparison (same 900 runs)	difference	only A / only B	McNemar p
offsets only vs trained fixed set	-2.4 ± 2.1	127 / 149	0.21
adaptive vs v12 set 2	+12.7 ± 2.3	219 / 105	2.3e-10

- **Adapting helps beyond tuning.** The fixed set had the same search, budget and all 7 parameters, and the adaptive policy still beats it by 6.9 points.
- **The gain is the adapting itself.** The policy's offsets without the feature terms are no better than the fixed set (−2.4, not significant).
- **Part of v13's earlier +16 points came from a weak base:** tuning the fixed set alone gained 5.8 points over v12 set 2.
- **Selection held up:** validation 0.730, test 0.714; the small drop is the usual effect of picking the best validation checkpoint.

9. Limits and next steps

- **α does not really adapt:** it sits at its 0.01 floor. Either the floor should be lowered, or α could be fixed and dropped from the adapted set.
- **Trained on 300 m only.** Its strongest settings (long look-ahead, sharp reactions) were tuned for the segment x ≈ 100 → 400 m. The 1000 m evaluation (v14/longroad) shows whether it carries over; if not, it should be trained on longer segments.
- **Unavoidable failures stay.** About 29 % of runs still fail; some are situations no parameter set can handle, where only something outside the potential (a safety filter, a learned acceleration correction) could help.
- **Small by design.** 40 weights, smooth and bounded, so it can be read and checked. Next steps if more is needed: adapt all 7 parameters, a tiny MLP trained the same way, or more features.
- **Repeatability.** Every run is exactly repeatable (SUMO launched without address randomisation and with a folder-independent command line), so all policies were compared on identical traffic.

10. Reproducing

step	command (v14/…)	time (7 jobs)
fixed set	`base: step2_train_base.py, step3_evaluate.py`	~6 h
adaptive	`adaptive: step2_train_policy.py, step3_evaluate.py, step4_analyze.py`	~7.5 h
1000 m test	`longroad: evaluate_longroad.py`	~35 min
watch a run	`adaptive: dipf_controller.py --episode … --policy policy.json --gui`	–